

M12 - Distributed Processing with Spark / PySpark

18-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Use Spark/PySpark safely with explicit schemas, plans, partitions, joins, shuffles and distributed validation.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M04-M05 Python and M11 storage/file-format foundations. You should understand dataframes conceptually, partitions, Parquet, joins, and validation.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE12-01 - Why Spark exists: driver, executors, partitions and tasks	5
DE12-02 - PySpark setup, SparkSession and laptop-safe local mode	7
DE12-03 - DataFrames and explicit schemas	10
DE12-04 - Spark SQL and expression equivalence	12
DE12-05 - Lazy transformations and actions	14
DE12-06 - Jobs, stages, tasks and physical plans	16
DE12-07 - repartition vs coalesce	18
DE12-08 - Aggregations and shuffle cost	20
DE12-09 - Joins and broadcast strategy	22
DE12-10 - Join correctness, fan-out and reconciliation	24
DE12-11 - Data skew	26
DE12-12 - Adaptive Query Execution (AQE)	28
DE12-13 - Caching and persistence	30
DE12-14 - Parquet I/O and partition pruning	32
DE12-15 - Built-in functions vs Python UDFs	34
DE12-16 - Driver safety: collect, toPandas and bounded inspection	36
DE12-17 - Testing and debugging Spark transformations	38
DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint	40

Prerequisite, tool and outcome map

ASSUMED

Complete M04-M05 Python and M11 storage/file-format foundations. You should understand dataframes conceptually, partitions, Parquet, joins, and validation.

MODULE PURPOSE

Use Spark/PySpark safely with explicit schemas, plans, partitions, joins, shuffles and distributed validation.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE12-01 - Why Spark exists: driver, executors, partitions and tasks

1. Why it matters

Explain Spark as a distributed execution engine and distinguish driver, executors, partitions, jobs, stages and tasks. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Explain Spark as a distributed execution engine and distinguish driver, executors, partitions, jobs, stages and tasks.

3. Prerequisite / what you already know

Complete M04-M05 Python and M11 storage/file-format foundations. You should understand dataframes conceptually, partitions, Parquet, joins, and validation.

4. Picture it in your head

Picture one coordinator with a job board and many workers holding boxes of data. The driver plans; partitions divide data; executors run tasks on those partitions.

5. New words before we use them

Spark exists because one process and one machine eventually become a bottleneck. The driver builds the application plan and coordinates work. Executors run tasks and hold data/cache for that application.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=FhaqbEOuQ8U)

URL: <https://www.youtube.com/watch?v=FhaqbEOuQ8U>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Sketch 16 orders split across four boxes. Mark one driver that plans a groupBy and four task boxes that process partitions. Then draw a shuffle before the final aggregate. Run the partition/skew simulator. Use it only to visualize uneven buckets; explicitly state that it is not Spark.

8. Expected result / what you should see

Your explanation keeps business row grain separate from physical partitions and says executors execute tasks while the driver coordinates the application.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: For Why Spark exists: driver, executors, partitions and tasks: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If everything feels like “a server,” start with one question: who plans, who executes, and what chunk of data does one task receive?

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one coordinator with a job board and many workers holding boxes of data. 3. Run the earliest independent control: For Why Spark exists: driver, executors, partitions and tasks: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Draw the driver/executor/partition/job/stage/task relationship from memory and explain what changes when partition count changes.

13. Validate your result

For Why Spark exists: driver, executors, partitions and tasks: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only.

14. Teach it back in your own words

Without notes, explain Why Spark exists: driver, executors, partitions and tasks in your own words. Start with the purpose: Explain Spark as a distributed execution engine and distinguish driver, executors, partitions, jobs, stages and tasks. Then name one realistic failure mode and explain how you would prove the result using this check: For Why Spark exists: driver, executors, partitions and tasks: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Why Spark exists: driver, executors, partitions and tasks with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-02.

DE12-02 - PySpark setup, SparkSession and laptop-safe local mode

1. Why it matters

Install/verify PySpark 4.2.0 safely and create a laptop-safe local SparkSession without treating local mode as a real cluster. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Install/verify PySpark 4.2.0 safely and create a laptop-safe local SparkSession without treating local mode as a real cluster.

3. Prerequisite / what you already know

Prerequisite: DE12-01 mastered with saved evidence.

4. Picture it in your head

SparkSession is the entry handle to Spark APIs. Local mode uses the same programming model on one machine, which is ideal for learning semantics without pretending your laptop is a cluster.

5. New words before we use them

PySpark is the Python API for Spark. For this course, first use is here in M12. Spark 4.2.0 supports Python 3.10+ and needs Java 17+.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Setup/orientation; use Teaching Book for laptop-safe setup

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=r_Sf6fCB40c)

URL: https://www.youtube.com/watch?v=r_Sf6fCB40c

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
Verify `java -version`;  
then `python -m pip install "  
pyspark==4.2.0"  
`;  
verify `python -c "  
import pyspark;  
print(pyspark.__version__)"  
`;
```

start a SparkSession with local[2], 2g driver memory and 4 shuffle partitions. Create a session named m12-check, print Spark version/master/shuffle partitions, then stop it cleanly.

8. Expected result / what you should see

Expected: Spark version 4.2.0, local[2], shuffle partitions 4. Exact JVM/OS text may differ.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For PySpark setup, SparkSession and laptop-safe local mode: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness de. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If Java errors appear, verify Java ≥ 17 and JAVA_HOME before reinstalling Spark. If import fails, check that ``python -m pip`` and ``python`` are the same environment.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: SparkSession is the entry handle to Spark APIs. 3. Run the earliest independent control: For PySpark setup, SparkSession and laptop-safe local mode: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Recreate the session from a blank file and explain every builder setting.

13. Validate your result

For PySpark setup, SparkSession and laptop-safe local mode: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain PySpark setup, SparkSession and laptop-safe local mode in your own words. Start with the purpose: Install/verify PySpark 4.2.0 safely and create a laptop-safe local SparkSession without treating local mode as a real cluster. Then name one realistic failure mode and explain how you would prove the result using this check: For PySpark setup, SparkSession and laptop-safe local mode: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness de.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for PySpark setup, SparkSession and laptop-safe local mode with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-03.

DE12-03 - DataFrames and explicit schemas

1. Why it matters

Build DataFrames with explicit schemas and explain why production ingestion should not blindly trust inference. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Build DataFrames with explicit schemas and explain why production ingestion should not blindly trust inference.

3. Prerequisite / what you already know

Prerequisite: DE12-02 mastered with saved evidence.

4. Picture it in your head

Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. Always know which room and dialect your query is using.

5. New words before we use them

A Spark DataFrame is distributed tabular data with named columns and types plus a query plan. Spark can infer CSV types, but inference costs work and can make contracts depend on sample data. An explicit StructType makes required types visible and repeatable.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ti3aC1m3rE8)

URL: <https://www.youtube.com/watch?v=ti3aC1m3rE8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use the provided order StructType in m12_reference_pipeline.py and read the normal orders.csv with header=true and no inferSchema. PrintSchema before any transformation. Create an explicit schema for the retest fixture, then identify which field is a business-validity flag rather than a type rule.

8. Expected result / what you should see

Normal orders: 16 rows, quantity 31, amount 3770.00. order_id/customer_id/region/category are strings; qty integer; amount double.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For DataFrames and explicit schemas: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If numeric columns arrive as strings, check whether an explicit schema was applied and whether malformed data was silently turned null. Inspect bad records rather than casting blindly.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. 3. Run the earliest independent control: For DataFrames and explicit schemas: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Close the guided code. From a blank file, define a schema for a fresh six-column shipment fixture, load it without inferSchema, and prove row count, types and one business-validity rule independently.

13. Validate your result

For DataFrames and explicit schemas: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain DataFrames and explicit schemas in your own words. Start with the purpose: Build DataFrames with explicit schemas and explain why production ingestion should not blindly trust inference. Then name one realistic failure mode and explain how you would prove the result using this check: For DataFrames and explicit schemas: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for DataFrames and explicit schemas with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-04.

DE12-04 - Spark SQL and expression equivalence

1. Why it matters

Prove equivalent results through DataFrame expressions and Spark SQL and explain when each style improves maintainability. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Prove equivalent results through DataFrame expressions and Spark SQL and explain when each style improves maintainability.

3. Prerequisite / what you already know

Prerequisite: DE12-03 mastered with saved evidence.

4. Picture it in your head

Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence.

5. New words before we use them

Spark SQL and the DataFrame API both describe logical transformations that Spark optimizes. Registering a DataFrame as a temp view lets SQL express the same grouped result as `groupBy().agg()`. Equivalence should be proved by result equality, not assumed because both queries look sensible.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=uAlp2VuZZPY)

URL: <https://www.youtube.com/watch?v=uAlp2VuZZPY>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

In the reference pipeline, compare the region aggregate built with `groupBy/agg` against the SQL query. Sort both before equality checking because distributed row order is not guaranteed without `ORDER BY`. Write the category total once with DataFrame functions and once with SQL, then compare sorted results.

8. Expected result / what you should see

Both routes must have three category rows and total amount 3770.00.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For Spark SQL and expression equivalence: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If equality fails, compare filters, null behavior, aggregation aliases and ordering. Do not blame Spark SQL before comparing logical expressions.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence. 3. Run the earliest independent control: For Spark SQL and expression equivalence: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Close both examples. Rebuild a two-column grouped result once with DataFrame expressions and once with Spark SQL on a changed category/zone grouping, normalize order, and prove exact equality.

13. Validate your result

For Spark SQL and expression equivalence: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain Spark SQL and expression equivalence in your own words. Start with the purpose: Prove equivalent results through DataFrame expressions and Spark SQL and explain when each style improves maintainability. Then name one realistic failure mode and explain how you would prove the result using this check: For Spark SQL and expression equivalence: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Spark SQL and expression equivalence with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-05.

DE12-05 - Lazy transformations and actions

1. Why it matters

Predict lazy execution and identify which operations are transformations versus actions. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Predict lazy execution and identify which operations are transformations versus actions.

3. Prerequisite / what you already know

Prerequisite: DE12-04 mastered with saved evidence.

4. Picture it in your head

Transformations build a plan without immediately processing all data; an action asks Spark for a result and triggers execution. Picture drawing the route before sending workers.

5. New words before we use them

Spark transformations describe new datasets without necessarily executing them immediately. Actions demand a result or side effect and trigger a job. Lazy planning lets Spark optimize across a chain before execution.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Transformations/actions sections + Teaching Book

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=r_Sf6fCB40c))

URL: https://www.youtube.com/watch?v=r_Sf6fCB40c

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Trace `orders.filter().select().groupBy().agg().`` and mark where no result is demanded. Then mark `count()` as the action that triggers execution. Write a five-step lineage with exactly one final action and predict when data is actually read.

8. Expected result / what you should see

Your classification must explain why printing a DataFrame object is not the same as materializing all its rows.

9. Why it worked

This works because predict lazy execution and identify which operations are transformations versus actions is made explicit before the result is accepted. Your independent check is: For Lazy transformations and actions: record the actual before/after state and verify the lesson requirement with an independent control. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If you are unsure, ask whether the call returns a new logical DataFrame or asks Spark to compute/return/write a result.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Transformations build a plan without immediately processing all data; an action asks Spark for a result and triggers execution. 3. Run the earliest independent control: For Lazy transformations and actions: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

From memory, write a new lazy lineage with at least four transformations and exactly two actions. Before running it, predict which lines trigger jobs and which only extend the plan; then verify on the learner runtime.

13. Validate your result

For Lazy transformations and actions: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Your classification must explain why printing a DataFrame object is not the same as materializing all its rows.

14. Teach it back in your own words

Without notes, explain Lazy transformations and actions in your own words. Start with the purpose: Predict lazy execution and identify which operations are transformations versus actions. Then name one realistic failure mode and explain how you would prove the result using this check: For Lazy transformations and actions: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Lazy transformations and actions with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-06.

DE12-06 - Jobs, stages, tasks and physical plans

1. Why it matters

Connect actions to jobs/stages/tasks and use EXPLAIN/UI evidence to reason about the physical plan. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Connect actions to jobs/stages/tasks and use EXPLAIN/UI evidence to reason about the physical plan.

3. Prerequisite / what you already know

Prerequisite: DE12-05 mastered with saved evidence.

4. Picture it in your head

Picture one coordinator with a job board and many workers holding boxes of data. The driver plans; partitions divide data; executors run tasks on those partitions.

5. New words before we use them

A logical query says what result is wanted. Spark produces optimized logical and physical plans describing how to execute it. An action creates one or more jobs; shuffle boundaries often create stage boundaries; tasks execute per partition.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=68CipcZt7ZA)

URL: <https://www.youtube.com/watch?v=68CipcZt7ZA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Call ``explain(mode="formatted"`

`)`` on a grouped DataFrame before the action. During a learner-machine run, open the Spark UI and relate the action to job/stage/task counts. Save one plan and annotate Scan, Exchange, Aggregate and any join node you see.

8. Expected result / what you should see

You can explain at least one cause of an Exchange and distinguish plan evidence from a tuning guess.

9. Why it worked

The reasoning separates correctness from performance: first prove the result, then inspect the planner/execution evidence to see whether the engine can avoid unnecessary work. Your independent check is: For Jobs, stages, tasks and physical plans: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If the plan is overwhelming, find the data source first, then work upward to filters, joins/exchanges and aggregates.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one coordinator with a job board and many workers holding boxes of data. 3. Run the earliest independent control: For Jobs, stages, tasks and physical plans: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Take a changed aggregation/join pipeline, save formatted EXPLAIN output, annotate scan, filter, exchange, join and aggregate nodes that actually appear, and connect one action to the observed job/stage/task evidence in the Spark UI.

13. Validate your result

For Jobs, stages, tasks and physical plans: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only.

14. Teach it back in your own words

Without notes, explain Jobs, stages, tasks and physical plans in your own words. Start with the purpose: Connect actions to jobs/stages/tasks and use EXPLAIN/UI evidence to reason about the physical plan. Then name one realistic failure mode and explain how you would prove the result using this check: For Jobs, stages, tasks and physical plans: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Jobs, stages, tasks and physical plans with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-07.

DE12-07 - repartition vs coalesce

1. Why it matters

Choose repartition or coalesce deliberately and explain the shuffle/cost trade-off. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Choose repartition or coalesce deliberately and explain the shuffle/cost trade-off.

3. Prerequisite / what you already know

Prerequisite: DE12-06 mastered with saved evidence.

4. Picture it in your head

NULL is an empty "unknown/not supplied" slot, not zero or an empty string. IS NULL tests that state; COALESCE chooses the first available fallback value deliberately.

5. New words before we use them

``repartition(n)`` can increase or decrease partitions and normally performs a shuffle to redistribute data. ``coalesce(n)`` is commonly used to reduce partitions without a full shuffle, trading balanced distribution for lower movement. Neither changes the business rows.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Partitions/shuffles context + Teaching Book

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=68CipcZt7ZA)

URL: <https://www.youtube.com/watch?v=68CipcZt7ZA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

On the normal DataFrame, record `getNumPartitions()`, then compare `repartition(4)`, `repartition(4,"region")` and `coalesce(1)` plans/partition counts. Write a decision table for increasing parallelism, reducing output files, key distribution and avoiding a single-partition bottleneck.

8. Expected result / what you should see

Every choice includes its intended downstream effect and a correctness control that row count/amount stay 16/3770.00.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For repartition vs coalesce: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If you only remember “coalesce is faster,” state whether you are increasing or decreasing partitions and whether balanced redistribution is required.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: NULL is an empty “unknown/not supplied” slot, not zero or an empty string. 3. Run the earliest independent control: For repartition vs coalesce: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

On changed data, choose between repartition and coalesce for two separate goals: balanced keyed work and fewer output files. Record before/after partition counts and defend why neither operation changes business grain.

13. Validate your result

For repartition vs coalesce: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain repartition vs coalesce in your own words. Start with the purpose: Choose repartition or coalesce deliberately and explain the shuffle/cost trade-off. Then name one realistic failure mode and explain how you would prove the result using this check: For repartition vs coalesce: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for repartition vs coalesce with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-08.

DE12-08 - Aggregations and shuffle cost

1. Why it matters

Measure aggregation shuffle cost and choose partition counts using evidence rather than copying defaults. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Measure aggregation shuffle cost and choose partition counts using evidence rather than copying defaults.

3. Prerequisite / what you already know

Prerequisite: DE12-07 mastered with saved evidence.

4. Picture it in your head

A distributed groupBy must bring rows with the same key together. That network movement is a shuffle, often more expensive than local per-partition work.

5. New words before we use them

Aggregations by key require records with the same key to meet, which commonly causes shuffle. Shuffle moves/serializes data and can create many intermediate blocks. ``spark.sql.shuffle.partitions`` defaults to 200 in Spark 4.2.0, which is intentionally too many for this tiny course dataset; the lab uses 4 to keep local work visible and light.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=9qrDsY_2COo)

URL: https://www.youtube.com/watch?v=9qrDsY_2COo

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Group orders by region and sum amount. Inspect the plan for Exchange. Compare the tiny lab with shuffle partitions 4 versus a deliberately larger value and observe task/partition differences on your machine. Explain why “200 is the Spark default” does not mean “200 is correct for every dataset.”

8. Expected result / what you should see

Region totals must sum back to 3770.00 and grouped row count is four.

9. Why it worked

The calculation is trustworthy because the output grain is declared before aggregation and the grouped result is reconciled to an independent detail-level control. Your independent check is: For Aggregations and shuffle cost: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared group. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If tuning discussion becomes guesswork, return to input size, partition sizes, task count, shuffle bytes and runtime evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A distributed groupBy must bring rows with the same key together. 3. Run the earliest independent control: For Aggregations and shuffle cost: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared group. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build a changed two-key aggregation, reconcile grouped totals back to source totals, then inspect the plan for Exchange/shuffle evidence. Explain why correctness proof must come before performance tuning.

13. Validate your result

For Aggregations and shuffle cost: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared group.

14. Teach it back in your own words

Without notes, explain Aggregations and shuffle cost in your own words. Start with the purpose: Measure aggregation shuffle cost and choose partition counts using evidence rather than copying defaults. Then name one realistic failure mode and explain how you would prove the result using this check: For Aggregations and shuffle cost: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared group.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Aggregations and shuffle cost with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-09.

DE12-09 - Joins and broadcast strategy

1. Why it matters

Choose join strategy with size/statistics awareness and use broadcast only when the small side is genuinely safe. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Choose join strategy with size/statistics awareness and use broadcast only when the small side is genuinely safe.

3. Prerequisite / what you already know

Prerequisite: DE12-08 mastered with saved evidence.

4. Picture it in your head

A large-large join usually requires data movement by key; broadcasting sends a small table to every executor so the large side can stay partitioned.

5. New words before we use them

Large joins often shuffle data so matching keys meet. If one side is small, Spark can broadcast it to executors and avoid shuffling the larger side. Spark 4.2.0 has a 10 MB automatic broadcast threshold by default for suitable plans, but hints are not unconditional guarantees.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=lu7TrqAWuH4)

URL: <https://www.youtube.com/watch?v=lu7TrqAWuH4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Join 16 orders to the 13-row customer dimension with `broadcast(customers)`. Inspect the formatted plan and look for the broadcast join operator on your learner runtime. Remove the explicit hint and compare the plan. Explain what statistics/threshold information influences automatic broadcast.

8. Expected result / what you should see

Correct join output remains 16 rows, amount 3770.00, and zero unmatched customer segments.

9. Why it worked

The relationship is trustworthy because the join key and expected cardinality are stated before interpreting the output, and row/total controls can expose fan-out or dropped unmatched rows. Your independent check is: For Joins and broadcast strategy: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If a broadcast hint looks ignored, check join type, plan, table size/statistics and Spark support before forcing another hint.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A large-large join usually requires data movement by key; broadcasting sends a small table to every executor so the large side can stay partitioned. 3. Run the earliest independent control: For Joins and broadcast strategy: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Join a changed fact fixture to a deliberately small unique dimension. Prove dimension-key uniqueness, compare an ordinary join with an explicit broadcast-hint plan, and save row/amount controls showing both are semantically identical.

13. Validate your result

For Joins and broadcast strategy: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

14. Teach it back in your own words

Without notes, explain Joins and broadcast strategy in your own words. Start with the purpose: Choose join strategy with size/statistics awareness and use broadcast only when the small side is genuinely safe. Then name one realistic failure mode and explain how you would prove the result using this check: For Joins and broadcast strategy: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Joins and broadcast strategy with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-10.

DE12-10 - Join correctness, fan-out and reconciliation

1. Why it matters

Detect fan-out and reconcile joins using row-count, key and measure controls. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Detect fan-out and reconcile joins using row-count, key and measure controls.

3. Prerequisite / what you already know

Prerequisite: DE12-09 mastered with saved evidence.

4. Picture it in your head

Reconciliation is like closing a cash register: independently compare row counts, keys, and important totals to the source before trusting a cleaned or summarized result.

5. New words before we use them

A join can run successfully and still multiply facts. If a supposed one-side dimension contains duplicate join keys, each matching fact can be duplicated. DISTINCT after the join hides symptoms and may delete legitimate repeated facts.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson + course reconciliation task

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=lu7TrqAWuH4)

URL: <https://www.youtube.com/watch?v=lu7TrqAWuH4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run `join_fanout_simulator.py`. Compare good and bad joins. Identify the duplicate dimension key and the inflated row/amount controls. In PySpark, load `customers_with_duplicate.csv`, `groupBy(customer_id).count().filter(count>1)`, then prove the join inflates before repairing the dimension contract.

8. Expected result / what you should see

Good join: 16 rows / 3770.00. The bad dimension must produce more rows and a larger amount than the source.

9. Why it worked

The relationship is trustworthy because the join key and expected cardinality are stated before interpreting the output, and row/total controls can expose fan-out or dropped unmatched rows. Your independent check is: For Join correctness, fan-out and reconciliation: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If a join has extra rows, do not add DISTINCT first. Test key uniqueness on each side and locate the key(s) with multiplicity.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Reconciliation is like closing a cash register: independently compare row counts, keys, and important totals to the source before trusting a cleaned or summarized result. 3. Run the earliest independent control: For Join correctness, fan-out and reconciliation: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a tiny duplicate-key dimension that causes fan-out. Demonstrate the wrong joined row count/measure, identify the duplicated key, repair the dimension contract, rerun, and prove the original fact totals are restored.

13. Validate your result

For Join correctness, fan-out and reconciliation: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation.

14. Teach it back in your own words

Without notes, explain Join correctness, fan-out and reconciliation in your own words. Start with the purpose: Detect fan-out and reconcile joins using row-count, key and measure controls. Then name one realistic failure mode and explain how you would prove the result using this check: For Join correctness, fan-out and reconciliation: Check pre/post row counts, key uniqueness on the expected "one" side, unmatched-key counts, and one important total before and after the relationship operation.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Join correctness, fan-out and reconciliation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-11.

DE12-11 - Data skew

1. Why it matters

Measure skew and explain why a few hot keys can dominate stage time. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Measure skew and explain why a few hot keys can dominate stage time.

3. Prerequisite / what you already know

Prerequisite: DE12-10 mastered with saved evidence.

4. Picture it in your head

Skew is an uneven workload: one partition gets a huge key and becomes the slow worker everyone waits for. Inspect key/partition distributions before tuning blindly.

5. New words before we use them

Skew means work is unevenly distributed: one or a few keys/partitions contain much more data than others. A job can appear mostly complete while one straggler task runs much longer. Measure key-frequency distribution and partition metrics before applying techniques such as salting or AQE skew handling.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Shuffle/skew context + Teaching Book

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=lu7TrqAWuH4))

URL: <https://www.youtube.com/watch?v=lu7TrqAWuH4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use the simulator region counts as a small visual. Then on Spark `groupBy(region).count()` and inspect max/min counts. The tiny course data is not a performance benchmark; it only teaches the diagnostic pattern. Create a changed in-memory DataFrame with one key repeated far more than others. Predict which partition/task becomes a straggler after hash redistribution.

8. Expected result / what you should see

Your evidence includes key-frequency counts and an explanation of why equal partition count does not guarantee equal partition size.

9. Why it worked

This works because measure skew and explain why a few hot keys can dominate stage time is made explicit before the result is accepted. Your independent check is: For Data skew: record the actual before/after state and verify the lesson requirement with an independent control. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If you cannot see skew, sort key counts descending and compare the largest key with median/typical keys.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Skew is an uneven workload: one partition gets a huge key and becomes the slow worker everyone waits for. 3. Run the earliest independent control: For Data skew: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a changed skewed-key fixture, calculate key-frequency concentration, predict the straggler key/partition risk, and propose one mitigation only after showing measured skew evidence.

13. Validate your result

For Data skew: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Your evidence includes key-frequency counts and an explanation of why equal partition count does not guarantee equal partition size.

14. Teach it back in your own words

Without notes, explain Data skew in your own words. Start with the purpose: Measure skew and explain why a few hot keys can dominate stage time. Then name one realistic failure mode and explain how you would prove the result using this check: For Data skew: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Data skew with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-12.

DE12-12 - Adaptive Query Execution (AQE)

1. Why it matters

Explain AQE and verify which adaptive features are enabled instead of treating it as magic auto-tuning. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Explain AQE and verify which adaptive features are enabled instead of treating it as magic auto-tuning.

3. Prerequisite / what you already know

Prerequisite: DE12-11 mastered with saved evidence.

4. Picture it in your head

AQE lets Spark adjust parts of the physical plan using runtime statistics, such as shuffle partition sizes or join strategy. It adapts execution, not business correctness.

5. New words before we use them

Adaptive Query Execution uses runtime statistics to revise parts of a SQL physical plan while the query runs. Spark 4.2.0 documents AQE as enabled by default. It can coalesce post-shuffle partitions, handle skewed shuffle partitions, and sometimes change join strategy.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Execution-plan context + Teaching Book AQE lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=68CipcZt7ZA)

URL: <https://www.youtube.com/watch?v=68CipcZt7ZA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Print `spark.sql.adaptive.enabled` and inspect a grouped/join plan. On a sufficiently meaningful learner dataset, compare the adaptive/final plan after an action. Write a three-column note: static plan assumption ->

runtime fact ->

possible AQE adaptation.

8. Expected result / what you should see

Your explanation names at least two AQE behaviors and says correctness controls remain unchanged.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For Adaptive Query Execution (AQE): Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If you describe AQE as “Spark automatically makes everything fast,” return to the exact documented adaptive rules and their configuration boundaries.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: AQE lets Spark adjust parts of the physical plan using runtime statistics, such as shuffle partition sizes or join strategy. 3. Run the earliest independent control: For Adaptive Query Execution (AQE): Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

On a changed join+aggregate query, record the AQE configuration, capture the initial/adaptive plan evidence available after an action, and state which adaptation Spark may make without claiming AQE fixes incorrect joins.

13. Validate your result

For Adaptive Query Execution (AQE): Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain Adaptive Query Execution (AQE) in your own words. Start with the purpose: Explain AQE and verify which adaptive features are enabled instead of treating it as magic auto-tuning. Then name one realistic failure mode and explain how you would prove the result using this check: For Adaptive Query Execution (AQE): Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Adaptive Query Execution (AQE) with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-13.

DE12-13 - Caching and persistence

1. Why it matters

Use cache/persist only for reused expensive data and prove unpersist lifecycle. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Use cache/persist only for reused expensive data and prove unpersist lifecycle.

3. Prerequisite / what you already know

Prerequisite: DE12-12 mastered with saved evidence.

4. Picture it in your head

Container writable layers are disposable. A volume is an external storage drawer mounted into the container so important state can outlive one container instance.

5. New words before we use them

Caching stores computed data so later actions can reuse it instead of rebuilding the same lineage. Cache has a memory/storage cost and can become stale relative to a changed external source. Cache when a reused dataset is expensive enough to justify storage; materialize it with an action, compare repeated-use evidence, and `unpersist()` when done.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Execution/storage context + Teaching Book caching lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=68CipcZt7ZA)

URL: <https://www.youtube.com/watch?v=68CipcZt7ZA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Create a filtered/derived DataFrame used by two actions. Cache it, materialize with count, inspect storage/UI, run the second action, then unpersist. Write a cache decision for three cases: reused expensive aggregate, one-time filter, tiny dimension broadcast.

8. Expected result / what you should see

You can state what is cached, why it is reused, how it is materialized, and when it is released.

9. Why it worked

This works because use cache/persist only for reused expensive data and prove unpersist lifecycle is made explicit before the result is accepted. Your independent check is: For Caching and persistence: record the actual before/after state and verify the lesson requirement with an independent control. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If caching seems to have no benefit, verify the same lineage is actually reused and compare runtime/storage metrics after materialization.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Container writable layers are disposable. 3. Run the earliest independent control: For Caching and persistence: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose one reused expensive lineage and one one-time lineage. Cache/materialize/unpersist only the reused one, leave the one-time lineage uncached, and save evidence supporting both decisions.

13. Validate your result

For Caching and persistence: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: You can state what is cached, why it is reused, how it is materialized, and when it is released.

14. Teach it back in your own words

Without notes, explain Caching and persistence in your own words. Start with the purpose: Use cache/persist only for reused expensive data and prove unpersist lifecycle. Then name one realistic failure mode and explain how you would prove the result using this check: For Caching and persistence: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Caching and persistence with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-14.

DE12-14 - Parquet I/O and partition pruning

1. Why it matters

Read/write Parquet with Spark and prove partition pruning without confusing directory partitions with Spark execution partitions. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Read/write Parquet with Spark and prove partition pruning without confusing directory partitions with Spark execution partitions.

3. Prerequisite / what you already know

Prerequisite: DE12-13 mastered with saved evidence.

4. Picture it in your head

Partition pruning skips directory partitions; Parquet column/predicate pruning skips unnecessary column chunks/row groups. Together they reduce bytes read before computation.

5. New words before we use them

Spark can read and write Parquet while preserving column types and using Parquet metadata. ``partitionBy("region")`` during write creates directory-style dataset partitions. A filter on a partition column can let Spark prune irrelevant directories.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: DataFrame I/O + Teaching Book Parquet/pruning lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ti3aC1m3rE8)

URL: <https://www.youtube.com/watch?v=ti3aC1m3rE8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

The reference pipeline writes normal orders partitioned by region, then reads back only East and selects `order_id/amount`. Inspect the output directories and formatted plan. Write a second Parquet dataset partitioned by category and compare the files/directories considered for a category filter.

8. Expected result / what you should see

Full read remains 16 rows / 3770.00. East filter returns only East rows. Explain both business and physical partition meanings.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For Parquet I/O and partition pruning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If pruning is absent, verify the filter targets an actual partition column and inspect PartitionFilters rather than guessing from runtime alone.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Partition pruning skips directory partitions; Parquet column/predicate pruning skips unnecessary column chunks/row groups. 3. Run the earliest independent control: For Parquet I/O and partition pruning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a changed Parquet dataset partitioned by a different low-cardinality column, read back a filtered subset, reconcile full and filtered controls, and inspect PartitionFilters/file-scan evidence for pruning.

13. Validate your result

For Parquet I/O and partition pruning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain Parquet I/O and partition pruning in your own words. Start with the purpose: Read/write Parquet with Spark and prove partition pruning without confusing directory partitions with Spark execution partitions. Then name one realistic failure mode and explain how you would prove the result using this check: For Parquet I/O and partition pruning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Parquet I/O and partition pruning with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-15.

DE12-15 - Built-in functions vs Python UDFs

1. Why it matters

Prefer Spark built-in expressions over Python UDFs when practical and explain the optimizer/serialization boundary. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Prefer Spark built-in expressions over Python UDFs when practical and explain the optimizer/serialization boundary.

3. Prerequisite / what you already know

Prerequisite: DE12-14 mastered with saved evidence.

4. Picture it in your head

A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change.

5. New words before we use them

Spark SQL built-in functions are visible to Spark's optimizer and execute within Spark's optimized expression engine. A Python UDF crosses a language/runtime boundary and can add serialization overhead while making expression semantics less transparent to Catalyst. UDFs are still useful when required logic has no suitable built-in; the rule is prefer built-ins, measure before optimizing, and test semantics.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Built-in expression style + Teaching Book UDF comparison

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=ti3aC1m3rE8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Create an uppercase/trimmed category with built-ins. Then write an equivalent simple Python UDF only for comparison and inspect plans/runtime on a learner machine. Find three transformations in your pipeline that should stay built-in and one hypothetical case that might justify a UDF.

8. Expected result / what you should see

Built-in and UDF versions must return equivalent values before any performance comparison.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting "the script ran" as correctness. Your independent check is: For Built-in functions vs Python UDFs: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If you reach for a UDF first, search current `pyspark.sql.functions` for an equivalent expression and compose built-ins where readable.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change. 3. Run the earliest independent control: For Built-in functions vs Python UDFs: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Without copying the guided example, implement one changed text/conditional transformation using built-in Spark functions and an equivalent Python UDF. Prove equal outputs first, then explain why built-ins remain the default choice.

13. Validate your result

For Built-in functions vs Python UDFs: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Built-in functions vs Python UDFs in your own words. Start with the purpose: Prefer Spark built-in expressions over Python UDFs when practical and explain the optimizer/serialization boundary. Then name one realistic failure mode and explain how you would prove the result using this check: For Built-in functions vs Python UDFs: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Built-in functions vs Python UDFs with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-16.

DE12-16 - Driver safety: collect, toPandas and bounded inspection

1. Why it matters

Protect the driver by bounding local collection and distinguishing debugging samples from distributed processing. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Protect the driver by bounding local collection and distinguishing debugging samples from distributed processing.

3. Prerequisite / what you already know

Prerequisite: DE12-15 mastered with saved evidence.

4. Picture it in your head

Picture one coordinator with a job board and many workers holding boxes of data. The driver plans; partitions divide data; executors run tasks on those partitions.

5. New words before we use them

``collect()`` returns all rows to the driver as Python objects. ``toPandas()`` also materializes data on the driver and should only be used when the result is small enough. Large unbounded collection can exhaust driver memory and defeats distributed processing.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Driver vs distributed work context + Teaching Book safety lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=r_Sf6fCB40c))

URL: https://www.youtube.com/watch?v=r_Sf6fCB40c

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use ``orders.limit(5).collect()`` for a tiny debug sample. Compare this with the unsafe habit of calling `collect` on an unknown large source. Rewrite three debugging patterns so each has an explicit bound or aggregated result before reaching the driver.

8. Expected result / what you should see

Every local-inspection action states an expected maximum row count and why it is safe on the learner laptop.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Driver safety: collect, toPandas and bounded inspection: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected con. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If you need “all rows to inspect,” define the question you are debugging and reduce columns/rows first.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one coordinator with a job board and many workers holding boxes of data. 3. Run the earliest independent control: For Driver safety: collect, toPandas and bounded inspection: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Rewrite three unsafe inspection requests into bounded driver-safe checks using limit/take/aggregates or writes. For each, state the maximum rows expected to reach the driver and why that bound is acceptable.

13. Validate your result

For Driver safety: collect, toPandas and bounded inspection: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Driver safety: collect, toPandas and bounded inspection in your own words. Start with the purpose: Protect the driver by bounding local collection and distinguishing debugging samples from distributed processing. Then name one realistic failure mode and explain how you would prove the result using this check: For Driver safety: collect, toPandas and bounded inspection: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected con.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Driver safety: collect, toPandas and bounded inspection with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-17.

DE12-17 - Testing and debugging Spark transformations

1. Why it matters

Test Spark transformations with deterministic fixtures and debug failures using schema + row + plan evidence. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Test Spark transformations with deterministic fixtures and debug failures using schema + row + plan evidence.

3. Prerequisite / what you already know

Prerequisite: DE12-16 mastered with saved evidence.

4. Picture it in your head

Test distributed transformations with small deterministic fixtures and invariants first; inspect schema/plan/counts so a cluster-scale run does not become the first correctness test.

5. New words before we use them

Spark code should be tested as transformations, not only by looking at notebook output. PySpark 4.2 provides testing utilities such as `assertDataFrameEqual` and `assertSchemaEqual`. Keep transformation functions small, feed deterministic fixtures, normalize order where order is not part of the contract, and assert schema plus business controls.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Use as visual refresher; testing task remains independent

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ti3aC1m3rE8)

URL: <https://www.youtube.com/watch?v=ti3aC1m3rE8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

After installing PySpark, run `test_transformations.py`. It builds a tiny DataFrame and checks the derived ``net`` column with `assertDataFrameEqual`. Write one test for a join that expects no fan-out and one test for a transformation that deliberately preserves null handling.

8. Expected result / what you should see

Tests are deterministic, order-aware where necessary, and fail when the intended contract is broken.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For Testing and debugging Spark transformations: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If equality fails for distributed results, first decide whether row order is part of the contract. Sort by a stable key before comparing when appropriate.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Test distributed transformations with small deterministic fixtures and invariants first; inspect schema/plan/counts so a cluster-scale run does not become the first correctness test. 3. Run the earliest independent control: For Testing and debugging Spark transformations: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a fresh deterministic unit test for a transformation not used in the bundled example. Include schema/business assertions and a deliberate failing case; save the failure and corrected pass.

13. Validate your result

For Testing and debugging Spark transformations: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain Testing and debugging Spark transformations in your own words. Start with the purpose: Test Spark transformations with deterministic fixtures and debug failures using schema + row + plan evidence. Then name one realistic failure mode and explain how you would prove the result using this check: For Testing and debugging Spark transformations: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Testing and debugging Spark transformations with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE12-18.

DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint

1. Why it matters

Build and defend an integrated local PySpark pipeline with explicit schema, quality controls, joins, aggregation, plan evidence, safe partitioning, Parquet output and tests. This matters because distributed code can return plausible results while partitions, shuffles and execution choices create correctness or scale failures.

2. What you are learning

Build and defend an integrated local PySpark pipeline with explicit schema, quality controls, joins, aggregation, plan evidence, safe partitioning, Parquet output and tests.

3. Prerequisite / what you already know

Prerequisite: DE12-17 mastered with saved evidence.

4. Picture it in your head

A checkpoint is a durable bookmark of progress. Replay safety means you can restart from a known checkpoint without losing data or creating duplicate side effects.

5. New words before we use them

The module checkpoint combines correctness and execution reasoning. Start with explicit schema and source controls. Transform with built-ins, validate joins before tuning, aggregate and reconcile measures, inspect plans for shuffle/join behavior, write partitioned Parquet, read it back with a pruning filter, and keep driver inspection bounded.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Review first-look + then build integrated pipeline independently

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=r_Sf6fCB40c)

URL: https://www.youtube.com/watch?v=r_Sf6fCB40c

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run the reference pipeline only after PySpark 4.2.0 is available. Then rebuild the same requirements from a blank file rather than copying it. Capture explain output and final controls. Build the changed Clinic Shipments retest only if assigned;

for normal mastery, construct your own equivalent pipeline over normal orders with one deliberate validation failure injected and repaired.

8. Expected result / what you should see

Normal integrated controls: 16 order rows, 31 quantity, 3770.00 amount, 16-row customer join, zero unmatched customer segments, four region groups.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: For Integrated PySpark pipeline and distributed-processing checkpoint: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a cor. The proof separates logical correctness from distributed execution/performance evidence.

10. Common beginner mistakes

If the integrated job fails, isolate in this order: environment/session -> schema/read -> quality -> join cardinality -> transformation -> action/write -> plan/tuning. Repair the smallest failed layer first.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A checkpoint is a durable bookmark of progress. 3. Run the earliest independent control: For Integrated PySpark pipeline and distributed-processing checkpoint: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

From blank files and the normal fixtures, build the full integrated M12 pipeline independently: explicit schema, validation, safe join, aggregation, plan evidence, partition decision, Parquet write/read, bounded inspection and tests. Do not copy the reference pipeline.

13. Validate your result

For Integrated PySpark pipeline and distributed-processing checkpoint: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain Integrated PySpark pipeline and distributed-processing checkpoint in your own words. Start with the purpose: Build and defend an integrated local PySpark pipeline with explicit schema, quality controls, joins, aggregation, plan evidence, safe partitioning, Parquet output and tests. Then name one realistic failure mode and explain how you would prove the result using this check: For Integrated PySpark pipeline and distributed-processing checkpoint: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a cor.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Integrated PySpark pipeline and distributed-processing checkpoint with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.